



◆ OFFENSIVE SECURITY SERIES ◆

ANDROID STATIC ANALYSIS

Decompilation, Secret Discovery & MobSF Scanning

jadx · apktool · MobSF · Hardcoded Secrets · Manifest · Crypto Audit

jadx

apktool

MobSF

Secrets

Manifest

Crypto

SQLi

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

ANDROID STATIC ANALYSIS — Decompilation, Secret Discovery & MobSF Scanning

01.	DECOMPILATION — JADX & APKTOOL
02.	HARDCODED SECRETS & INSECURE CODE
03.	MOBSF AUTOMATED SCAN
04.	ANDROID STATIC ANALYSIS CHEAT SHEET



01 — DECOMPILED

jadx + apktool

```
jadx + apktool Decompileation

# Decompile APK with jadx
(kali)-[~]
kali $ jadx bankingapp.apk -d output_java/
INFO - loading ...
INFO - processing ...
INFO - done

# Decompile to smali with apktool
(kali)-[~]
kali $ apktool d bankingapp.apk -o decompiled/
I: Using Apktool 2.9.3 on bankingapp.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmaling classes.dex...
I: Finished

# Check manifest for dangerous flags
(kali)-[~]
kali $ grep -E "debuggable|allowBackup|exported" decompiled/AndroidManifest.xml
    android:debuggable="true"
    android:allowBackup="true"
    android:exported="true" <!-- LoginActivity -->
    android:exported="true" <!-- ResetPasswordActivity -->
```

Figure: APK decompiled, manifest shows debuggable=true, 2 exported activities without permissions



02 — SECRET DISCOVERY

Grep + Code Review

```

Static Analysis – Hardcoded Secrets & Insecure Code

# Search for hardcoded secrets in decompiled Java
(kali)-[~]
kali $ grep -rn "api_key\\|secret\\|password\\|token\\|bearer" output_java/sources/ -i --include="*.java" | grep -v "//\\|
import" | head -15
output_java/sources/com/target/app/BuildConfig.java:9: public static final String API_KEY = "sk-live-Abc123XY
Zdef456GHI";
output_java/sources/com/target/app/BuildConfig.java:10: public static final String BASE_URL = "https://api.tar
get.com/v2/";
output_java/sources/com/target/app/NetworkManager.java:45: private static final String SECRET = "jwt_signing_s
ecret_2024_prod";
output_java/sources/com/target/app/DatabaseHelper.java:12: private static final String DB_PASS = "DbP@ss2024!"
;

# Search for insecure crypto
(kali)-[~]
kali $ grep -rn "AES/ECB\\|MD5\\|new Random()\\|DESede" output_java/sources/ --include="*.java"
output_java/sources/com/target/app/CryptoUtils.java:23: Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Paddi
ng");
output_java/sources/com/target/app/AuthManager.java:67: MessageDigest md = MessageDigest.getInstance("MD5");

# Find raw SQL queries (SQLi potential)
(kali)-[~]
kali $ grep -rn "rawQuery\\|execSQL" output_java/sources/ --include="*.java"
output_java/sources/com/target/app/UserRepository.java:34: db.rawQuery("SELECT * FROM users WHERE id=" + userI
d, null);

```

Figure: Live API key, JWT secret, DB password, AES/ECB mode and rawQuery SQLi all found



03 — MOBSF SCAN

Automated Static Analysis

```
MobSF API — Automated Scan Results

# Run MobSF automated static scan via API
(kali)-[~]
kali $ curl -F "file=@bankingapp.apk" http://127.0.0.1:8000/api/v1/upload -H "Authorization: RTG_API_KEY"
{"scan_type": "apk", "hash": "alb2c3d4e5f67890abcdef12345678ab", "file_name": "bankingapp.apk", "analyzer": "static_analyzer", "status": "success"}

(kali)-[~]
kali $ curl -s "http://127.0.0.1:8000/api/v1/report_json?hash=alb2c3d4e5f67890abcdef12345678ab" -H "Authorization: RTG_API_KEY" | python3 -m json.tool | grep -A2 "\"level\": \"high\""
  "title": "AES ECB Mode",
  "level": "high",
  "description": "AES ECB mode is weak and should not be used",
  "title": "Hardcoded API Key",
  "level": "high",
  "description": "API key found hardcoded in BuildConfig",
  "title": "SQL Injection Risk",
  "level": "high",
  "description": "Unsafe.rawQuery() usage with string concatenation"
```

Figure: MobSF returns 3 high severity findings — AES/ECB, hardcoded key, SQL injection



04 — CHEAT SHEET

Quick Reference

jadx decompile	<code>jadx target.apk -d output/</code>
apktool decompile	<code>apktool d target.apk -o decompiled/</code>
Check debuggable	<code>grep -i debuggable decompiled/AndroidManifest.xml</code>
Check allowBackup	<code>grep -i allowBackup decompiled/AndroidManifest.xml</code>
Find exported	<code>grep -B2 -A5 "exported=\"true\"" decompiled/AndroidManifest.xml</code>
Find secrets	<code>grep -rn "api_key\ secret\ password\ token" output/sources/ -i --include="*.java"</code>
Find endpoints	<code>grep -rn "https?:\/\/\ BASE_URL" output/sources/ --include="*.java" sort -u</code>
Find crypto issues	<code>grep -rn "AES/ECB\ DES\ MD5\ new Random()" output/sources/ --include="*.java"</code>
Find SQLi	<code>grep -rn "rawQuery\ execSQL" output/sources/ --include="*.java"</code>
Find WebView issues	<code>grep -rn "setJavaScriptEnabled\ addJavascriptInterface" output/sources/ --include="*.java"</code>
TruffleHog scan	<code>trufflehog filesystem output/ --only-verified</code>
Semgrep Android	<code>semgrep --config p/android output/sources/</code>
MobSF upload	<code>curl -F "file=@app.apk" http://127.0.0.1:8000/api/v1/upload -H "Authorization: KEY"</code>
Resign patched APK	<code>apktool b decompiled/ -o patched.apk && uber-apk-signer -a patched.apk</code>
Find native strings	<code>strings decompiled/lib/arm64-v8a/*.so grep -iE "key secret http"</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.